JOURNAL OF
Optical Communications
and Networking

DTPE: a distributed topology perturbation engine for tail-risk mitigation in degree-constrained optical DCNs

Xinyue Gu,[1] Hao Yang,[1,*] and Zuqing Zhu[2]

[1]Department of Information and Control Engineering, Southwest University of Science and Technology, Mianyang, Sichuan 621010, China
[2]School of Information Science and Technology, University of Science and Technology of China, Hefei, Anhui 230027, China
*yh@swust.edu.cn

Received 13 January 2026; revised 23 March 2026; accepted 9 April 2026; published 18 May 2026

Reconfigurable optical data-center networks (DCNs) often suffer from a fundamental timescale mismatch: while the baseline topology is planned over coarse epochs (e.g., minutes), practical workloads generate bursty, skewed demands that cause severe tail congestion within seconds. To bridge this gap, we propose DTPE, a distributed topology perturbation engine that operates continuously alongside the baseline. Instead of triggering global reconfigurations, DTPE empowers rack-level agents to negotiate local, degree-budget-aware circuit trades using a small amount of reserved port slack. This allows the network to absorb emerging hotspots through rapid, adjacency-preserving perturbations without involving the central controller. Extensive simulations across varying traffic patterns and planning algorithms (greedy, demand-gravity, and Couder) demonstrate that DTPE effectively suppresses the maximum-link-utilization (MLU) tail. Specifically, it reduces the overload probability by orders of magnitude (e.g., from ∼12.2% to below 1.0% for $N = 32$) while scaling robustly under realistic degree constraints. Finally, we validate the physical feasibility of the proposed scheme via a trace-driven replay on a four-rack OCS testbed, confirming that such fine-grained reconfiguration actions can be executed on real hardware to mitigate congestion delays.

https://doi.org/10.1364/JOCN.590053

1. INTRODUCTION

Modern machine-learning and data-analytics workloads generate traffic patterns that are far from smooth. They mix long-lived "elephant" transfers with short, bursty exchanges that appear and disappear on the order of hundreds of milliseconds. Packet-switched data center networks (DCNs) can keep up with the small bursts, but often struggle to provide a predictable completion time for large jobs. Reconfigurable all-optical DCNs promise another degree of freedom: they can reshape their topology at run time and steer more capacity toward hotspots [1–4]. The challenge is to decide *when* and *how* to rewire.

Early optical DCN proposals, such as Helios and c-Through, focused on integrating optical circuit switches into otherwise static multi-tier topologies and on demonstrating basic feasibility [5,6]. Later work pushed toward fully reconfigurable optical fabrics, including OSA and Hyper-FleX-LION, where the physical topology can be reshaped to match the current traffic matrix [2,7,8]. Meanwhile, WAN TE systems such as B4 and SWAN [9,10] demonstrated that centralized controllers can periodically compute near-optimal routing or topology plans based on an aggregated view of demand [9,10]. Most of these systems, however, operate on timescales of seconds or more. In the prevailing epoch-based planning paradigm, the centralized planner deliberately amortizes optimization and reconfiguration over a longer window; localized bursts that unfold *within* an epoch are therefore unlikely to trigger a global recomputation and can still create transient hot links and inflate tail metrics even when the baseline is globally reasonable.

Our earlier work on Hyper-FleX-LION and HOE-DCN took a fairly centralized view of this problem [8,11]. We formulated large integer linear programs (ILPs) for topology planning and, in some cases, bilevel models to capture the interaction between topology remapping and virtual network embedding [11–13]. These formulations give us a clean notion of an "optimal" topology for a given long-term traffic matrix, but they are hard to solve online at scale. At the same time, they are also not designed to react to localized demand deviations: a short-lived burst between a small subset of racks does not necessarily justify re-running a global traffic-aware optimizer (ILP or heuristic) and reconfiguring the circuit fabric at the same cadence.

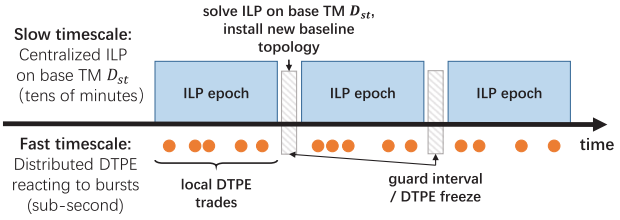

Fig. 1. Two timescales for topology planning and adaptation. Top: a centralized planner computes a baseline topology for long-term demands (D_{st}). Bottom: distributed agents continuously adapt the topology to transient bursts using reserved slack ports.

In this work, we revisit topology planning for reconfigurable optical DCNs from a more pragmatic angle. We intentionally separate the problem into two timescales. At a coarse timescale (tens of seconds or minutes), a centralized planner computes a topology that is good for a *long-term base* traffic matrix—an aggregated rack-to-rack demand D_{st}. At a finer timescale (a few seconds, bounded by the monitoring window and OCS reconfiguration latency), lightweight distributed agents at the racks are allowed to make small, negotiated rewiring moves in response to local bursts. Most of the time, the base topology stays in place; the distributed scheme only nudges it when persistent hotspots appear.

As sketched in Fig. 1, the baseline topology is obtained from a centralized traffic-aware optimizer that minimizes the maximum link utilization (MLU) under the base matrix D_{st}. We use this optimizer as a *structural oracle* rather than an online engine. Its solution sets the stage: it tells us how a demand-aware optical DCN would look if we only cared about long-term averages. On top of that, distributed agents occasionally establish or reassign a small number of circuits near hot links to absorb short-term deviations. In our experiments, we evaluate the impact of these choices on both MLU and job completion time (JCT); MLU is not a perfect surrogate for JCT, but it remains a widely used proxy for congestion risk and load balancing in data-center workloads [14,15].

Even if a centralized heuristic is computationally light, the end-to-end reaction time of a fast control loop is dominated by the *control plane*: (i) aggregating timely rack-level utilization and demand signals, (ii) computing a globally consistent set of port-feasible topology edits under per-rack degree budgets, and (iii) disseminating and sequencing OCS commands. This fan-in/fan-out loop grows with network scale and must be repeated whenever bursts shift; for short-lived bursts that unfold within a planning epoch, this round-trip latency can become comparable to the burst lifetime, undermining timely mitigation. Moreover, relying on a central entity for micro-adjustments risks control-plane saturation and introduces a single point of bottleneck during bursty episodes. DTPE circumvents this global loop by confining signaling to local neighbors, ensuring that reaction time is dominated mainly by link-local control-plane RTT and remains essentially constant with N, while providing continuous tail protection between slow ILP epochs.

To prevent oscillation or routing loops without global coordination, DTPE employs a localized "elementary rewiring move" and a cooperative acceptance rule grounded in a benefit–cost model.

A. Our Scope and Contributions

The focus of this paper is on the *interplay* between a centralized, ILP-based planner and a lightweight distributed rewiring scheme in an all-optical DCN. We adopt an abstract reconfigurable topology model, rather than committing to a specific optical switch architecture. This allows us to discuss Hyper-FleX-LION, OSA-like AOI fabrics, and other designs within a single framework [1,2,7,8].

At a high level, our main contributions are as follows:

- We formulate an ILP for centralized topology planning on an aggregated rack-to-rack traffic matrix under per-rack degree constraints and introduce a slack parameter σ that reserves a small port budget for later distributed rewiring.
- We design DTPE, a distributed perturbation engine in which rack-level agents monitor local link utilization and negotiate rewiring moves using a shared benefit–cost acceptance rule, without involving the central controller.
- We evaluate DTPE across three baseline planning directions (greedy, gravity, and Couder [16]) and a broad range of traffic patterns and network scales, showing consistent reductions in MLU tail risk under realistic degree constraints.

The remainder of the paper is organized as follows: Section 2 introduces our abstract DCN model, traffic assumptions, and the ILP formulation. Section 3 describes the distributed rewiring scheme in detail. Section 4 presents the evaluation results, and Section 5 concludes.

B. Related Work

1. Optical and Reconfigurable DCN Architectures

A long line of work has explored hybrid electronic/optical and all-optical DCN architectures, including Helios, c-Through, OSA, and several later designs that rely on wavelength-routed AOIs [17] or fast optical circuit switches [1,2,5,6,18]. More recent surveys discuss reconfigurable optical networks in both DCN and WAN settings and summarize technological enablers and algorithmic challenges [3,4,19]. Our work does not propose yet another physical topology; instead, we adopt a generic degree-constrained multigraph model that can represent many of these architectures at the rack level.

2. Topology Planning and Traffic Engineering

Centralized TE systems such as B4 and SWAN demonstrated that a logically centralized controller can leverage traffic forecasts and periodically optimize WAN topologies or routing to achieve high utilization while meeting service-level objectives [9,10]. In the optical DCN domain, several works have proposed topology planning and engineering approaches for reconfigurable DCNs [16,20–22], including our own prior work on Hyper-FleX-LION and HOE-DCN [7,8,11]. Different from these prior studies that compute an end-to-end topology on a relatively slow timescale, DTPE adds a separate fast-timescale distributed perturbation layer that reuses a given baseline, consumes only a small slack-port budget, and is evaluated primarily on tail-risk metrics under bursty demand. These models are valuable as references, but they are typically too heavy for second-level closed-loop control within an epoch.

Our ILP baseline follows the same tradition but is explicitly positioned as a long-term structural oracle that operates on an aggregated base traffic matrix.

3. Scheduling and Bursts

At the flow and coflow level, many algorithms target job completion time under fixed topologies, e.g., Varys, coflow scheduling with improved bounds, and later variants [14,15]. For optical DCNs in particular, Tang *et al.* survey scheduling and *-flow management in OCS-based networks, emphasizing the impact of reconfiguration time on performance [19]. Our work sits one level below these schedulers: we assume a given traffic matrix and study how the physical topology can adapt at two timescales, especially in the presence of bursts that are long enough for an optical reconfiguration to be worthwhile.

2. SYSTEM MODEL AND ILP BASELINE

This section formalizes the abstract DCN model used in the rest of the paper and presents the ILP that plays the role of a centralized baseline. We focus on a single all-optical interconnect fabric and do not model the packet-switching layer explicitly; it is assumed to provide fall-back connectivity when some rack pairs are not directly connected by optical circuits.

A. Reconfigurable Rack-Level Topology

We model the DCN at the granularity of top-of-rack (ToR) switches. Let $\mathcal{V}$ denote the set of racks, with $|\mathcal{V}| = N$. The optical fabric can establish full-duplex circuit bundles between racks, subject to per-rack degree constraints. We represent the *configurable* rack-pair feasibility by a complete undirected graph on $\mathcal{V}$: any unordered pair $\{i, j\}$ can be connected by configuring the OCS, while the *realized* optical topology at any instant is the subgraph induced by pairs with $m_{ij} > 0$.

Formally, let $m_{ij} \in \mathbb{Z}_{\geq 0}$ denote the number of full-duplex circuit pairs between racks i and j, for $i \neq j$. Each circuit provides a fixed capacity C in each direction (e.g., a wavelength channel at 100 Gb/s). Define the *realized* optical degree (i.e., the number of active incident circuits) of rack i as $d_i \triangleq \sum_{j \neq i} m_{ij}$, which satisfies $0 \leq d_i \leq d_i^{\max}$, where $d_i^{\max}$ is the physical port budget of rack i; in general, the inequality need not be tight, e.g., when ports remain unused or are intentionally reserved as slack.

Various physical implementations map to this abstract model. For example, in Hyper-FleX-LION, circuits correspond to optical connections through AOIs and arrayed-waveguide grating routers; in OSA-like designs, they correspond to paths through wavelength-selective switches [2,7,8]. In both cases, the rack-level degree constraint captures the limited number of transceivers and optical ports per ToR.

Figure 2 shows a small illustrative example with $N = 5$ racks and a handful of circuits. While the physical realization remains abstract, the model explicitly captures rack-pair connectivity and circuit counts.

B. Traffic Model and Time Scales

We distinguish between two types of traffic information.

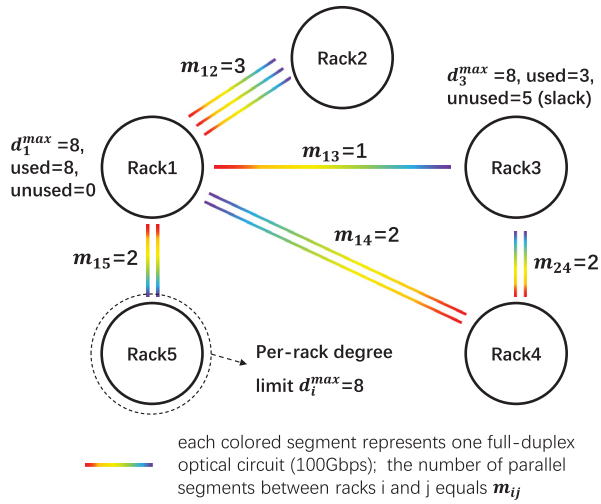

Fig. 2. Abstract model of a reconfigurable optical DCN. Racks are connected by bundles of full-duplex optical circuits (m_{ij}), subject to physical port constraints ($d_i^{\max}$) shown as node capacities.

- A *long-term base* traffic matrix D_{st}, which represents the average or percentile demand from rack s to rack t over a coarse planning window (tens of seconds or minutes). This is the input to the centralized ILP.
- Short-term *deviations* around this base, which we refer to as bursts. These bursts are observed locally by rack-level agents and trigger distributed rewiring in DTPE.

The ILP baseline ignores these bursts; folding them into D_{st} as peak values would force global over-provisioning while potentially missing shifting hotspots. Instead, base traffic reflects what the operator considers stable enough to justify a relatively expensive re-optimization of the topology. The distributed scheme, in contrast, explicitly targets these deviations. In practice, we view D_{st} as capturing the dominant long-lived demand components (e.g., persistent coflow/shuffle phases or heavy-hitter rack pairs) that are stable enough to justify an ILP re-optimization at the coarse timescale, rather than an all-to-all background. Operationally, D_{st} can be obtained by aggregating a small number of communication clusters over the planning window, while the residual low-volume and diffuse traffic is assumed to be handled by the packet-switched fall-back layer and therefore does not drive the optical baseline topology. This abstraction keeps the ILP from over-fitting to ephemeral, spatially spread bursts and aligns with our two-time-scale separation.

We assume that the optical fabric has a non-negligible reconfiguration timescale, so burst-induced hotspots may persist within an ILP epoch and motivate lightweight within-epoch adaptation.

C. ILP Formulation for the Baseline Topology

We now describe the ILP that computes a baseline topology for the base traffic matrix D_{st}. The ILP minimizes the maximum link utilization across all optical circuits, under per-rack degree constraints and per-circuit capacity limits.

Let $f_{ij}^{st} \geq 0$ denote the amount of base traffic from rack s to rack t that is routed over the directed arc (i, j). For each directed arc, the total flow over all commodities cannot exceed the installed capacity multiplied by a scalar θ that upper-bounds link utilization:

$$\sum_{s,t \in \mathcal{V}} f_{ij}^{st} \leq \theta\, m_{ij} C, \quad \forall \quad (i, j),\ i \neq j, \tag{1}$$

where θ is a decision variable representing the MLU of the baseline topology.

Flow conservation holds for each commodity (s, t) at each intermediate rack:

$$\sum_{j \in \mathcal{V} \setminus \{i\}} f_{ij}^{st} - \sum_{j \in \mathcal{V} \setminus \{i\}} f_{ji}^{st} = \begin{cases} D_{st}, & i = s, \\ -D_{st}, & i = t, \\ 0, & \text{otherwise}, \end{cases} \tag{2}$$

for all $i, s, t \in \mathcal{V}$ with $s \neq t$.

The degree constraint at each rack i is

$$\sum_{j \in \mathcal{V} \setminus \{i\}} m_{ij} \leq (1 - \sigma) d_i^{\max}, \quad \forall i \in \mathcal{V}, \tag{3}$$

where $0 \leq \sigma < 1$ is a *slack factor*. When $\sigma = 0$, the ILP is free to consume all available ports. When $\sigma > 0$, a fraction $\sigma d_i^{\max}$ of the ports at each rack are intentionally left unassigned in the baseline topology. These reserved ports form a small dynamic pool that the distributed agents can use for temporary circuits during bursts. In Fig. 2, for example, rack 3 uses only three out of eight degree units, leaving five slack ports in this dynamic pool.

Finally, the ILP bounds all variables:

$$m_{ij} \in \mathbb{Z}_{\geq 0}, \quad f_{ij}^{st} \geq 0, \quad \theta \geq 0. \tag{4}$$

The objective is to minimize θ:

$$\min \theta \tag{5}$$

subject to constraints Eqs. (1)–(3). This is a standard multi-commodity flow formulation with additional degree constraints and is NP-hard in general. In our setting, we solve it exactly for small topologies to obtain a ground-truth baseline and use heuristic solvers for larger instances.

D. Interpretation of the Slack Factor

The slack factor σ does not introduce a new theoretical construct but encodes a common operator policy: reserving headroom to absorb demand fluctuations. The difference is that here the headroom appears in the space of optical ports instead of link capacities.

Reserving slack ports imposes a cost: it reduces the static degrees available to the baseline planner, potentially impacting steady-state utilization. However, real optical DCNs often keep a small amount of headroom anyway to absorb traffic prediction error, transient demand, maintenance events, and incremental expansion. DTPE makes this headroom an explicit, tunable budget and quantifies its dynamic value in terms of tail-risk reduction (rather than average-case throughput).

Two regimes are most relevant:

- $\sigma = 0$. The baseline planner may consume all ports, leaving no pre-reserved headroom. DTPE can still act, but a rack must first tear down an existing circuit to free a port for a temporary rewire.
- $\sigma > 0$. The baseline planner is forced to leave some ports unused as explicit headroom that DTPE can exploit for fast rewiring. Increasing σ trades static capacity for dynamic flexibility, and our σ-sweep results (Fig. 8 below) show diminishing marginal returns beyond a small number of reserved ports.

In all cases, the baseline planner (an ILP when tractable or a traffic-aware heuristic in our evaluation) still targets min-MLU under the reduced per-rack budget induced by σ, while DTPE operates within the resulting slack headroom using the integer-rounding rule in our simulator.

3. DISTRIBUTED TOPOLOGY PERTURBATION ENGINE (DTPE)

The centralized ILP in Section 2.C provides a long-term baseline optical topology that is well matched to an aggregated rack-to-rack traffic matrix. In practice, however, traffic on top of this baseline still exhibits bursts and skews. Re-optimizing the ILP for every burst is not realistic. Instead, we introduce a lightweight *distributed topology perturbation engine* (DTPE) that runs at rack level between ILP epochs and performs small topology adjustments around persistent hot links. DTPE has three key properties. First, it only uses local information: per-link utilization measurements, the current port usage of each rack, and a small amount of state exchanged with immediate neighbors. Second, by design, it only reacts to bursts on *existing* optical links; it does not attempt to create completely new rack pairs that were disconnected in the ILP baseline. Third, it uses a unified *benefit–cost bilateral trading model*: each hot link between two racks is treated as a potential trade, where both endpoints decide whether the reduction in congestion justifies freeing an additional port at each end.

A. Rack-Level Agents, Visibility, and Control Plane

We assume that each rack $v \in \mathcal{V}$ hosts a lightweight DTPE software agent co-located with the ToR switch (see Fig. 3). The agent can read local statistics and issue control commands, but it does not participate in global optimization and does not maintain a full view of the network.

1. Local View and Visibility

For each incident full-duplex optical link $e = (v, x)$ (a bundle of m_{vx} circuit pairs), the agent at rack v maintains (i) byte counters or rate samples for traffic on e, from which it computes a sliding-window utilization $\bar{\rho}_e(\Delta t)$; (ii) the number of currently used optical ports at v; and (iii) the number of remaining slack ports, i.e., degree units not used by the ILP baseline (the "unused" part illustrated for rack 3 in Fig. 2). The agent does not know the full end-to-end traffic matrix or which remote rack pairs are directly connected. This L1-style visibility is deliberate: DTPE perturbs the baseline multigraph using

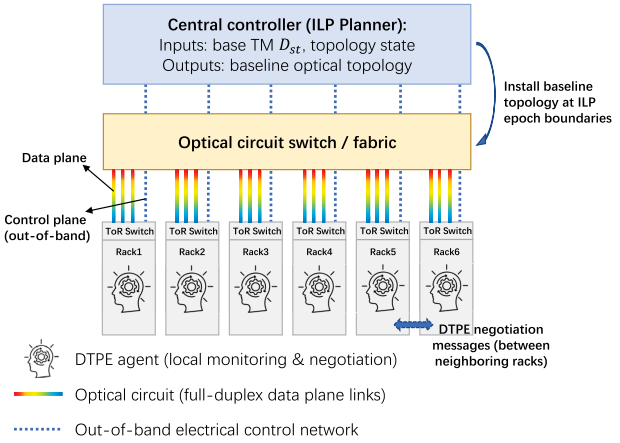

Fig. 3. DTPE control architecture. A centralized planner installs baseline topologies at epoch boundaries, while rack-level agents negotiate local adjustments over an out-of-band control plane (blue dashed lines) to modify the optical data plane (colored lines).

only per-link information, while the question of whether to introduce entirely new rack pairs is left to the slower centralized planner.

2. Actions

Given this local view, the DTPE agent at v can (i) mark certain incident links as *hot link candidates* when their utilization crosses a threshold for a sustained period, (ii) exchange small state messages with the DTPE agent at the other endpoint of a hot link (e.g., current utilization and local port prices), and (iii) trigger changes in the optical topology by requesting the optical circuit switch (OCS) to tear down or establish circuits.

3. Control Plane

We assume an always-on out-of-band electrical control network, as sketched in Fig. 3. All DTPE signaling between neighboring agents and all commands from the centralized controller to the optical circuit switch travel over this electrical control plane. They never consume bandwidth on the data-plane optical circuits that DTPE is reconfiguring, and the volume of such messages is small compared to data traffic: each bilateral negotiation exchange carries at most a handful of scalar values (link utilizations and port prices, per Algorithm 2 below), amounting to tens of bytes per trade, which is negligible relative to the 100 Gb/s optical data-plane circuits.

4. Timescale Relative to the ILP

DTPE runs continuously at the monitoring-slot timescale within each ILP epoch. Near the end of an epoch, a short guard interval freezes DTPE and allows any in-flight reconfigurations to finish. The controller then installs a fresh ILP baseline topology. Any outstanding temporary perturbations are reconciled (rolled back or overwritten) to restore consistency before DTPE resumes in the next epoch.

Figure 3 summarizes this control loop and the interaction between the centralized planner and rack-level agents.

B. Design Rationale: Adjacency Preservation Constraint

DTPE is designed only to *rewire* circuits among already-adjacent racks and never creates new optical neighbors. This is an explicit design choice for two reasons.

First, it follows from our visibility and signaling model. DTPE agents observe only local link utilization on incident circuits and exchange lightweight messages only with their current neighbors over the out-of-band control plane. Creating a new adjacency would require global discovery (who is a good new neighbor) and global coordination (how to preserve feasibility across many simultaneous changes), which substantially increases complexity and defeats DTPE's goal of fast, purely local reaction.

Second, creating new neighbors constitutes a structural topology change that couples to global routing and connectivity. Altering adjacency relations can invalidate baseline routing assumptions, interact with failure-handling and ECMP policies, and, in the worst case, risk disconnecting parts of the graph if done without a global view. We, therefore, reserve adjacency evolution for the slow-time-scale planner, which can reason over the network-wide traffic matrix and constraints when installing the next baseline topology. DTPE focuses on fast perturbations that preserve baseline adjacency while mitigating transient hotspots within the reserved slack budget.

This design intentionally scopes DTPE to handle hotspots that arise on existing adjacencies within an ILP epoch; hotspot pairs requiring new adjacencies are handled by the coarse-timescale planner at the next epoch boundary, which has a global view of the traffic matrix. Empirically, Section 4 shows that this adjacency-preserving constraint still allows DTPE to deliver substantial tail-risk mitigation under bursty demand.

C. Local Monitoring and Burst Detection

DTPE relies on per-link utilizations rather than on a global, instantaneous traffic matrix. For each full-duplex optical link $e = (i, j)$ that currently has at least one circuit, the DTPE agents at i and j maintain a sliding-window estimate $\bar{\rho}_e(\Delta t)$. The window size Δt is chosen to smooth out short-lived fluctuations but still capture bursts whose duration is comparable to the optical reconfiguration latency τ_{reconf}.

We declare a link e to be a *hot link* when the following conditions hold:

$$\bar{\rho}_e(t') > \theta_{\text{hot}} \quad \forall\, t' \in \{t - (K_{\text{hot}} - 1)\Delta t, \ldots, t\}; \tag{6}$$

that is, link e has exceeded the hot threshold of K_{hot} consecutive monitoring windows ending at time t. Where $\theta_{\text{hot}} \in (0, 1)$ is a utilization threshold and K_{hot} is a minimum persistence length (in windows). These parameters filter out very short spikes and measurement noise. Only such sustained hot links become candidates for DTPE trades. Algorithm 1 summarizes the local monitoring and hot-link detection procedure.

D. Benefit–Cost Model for a Single Hot Link

We now describe how DTPE decides whether and how to react to a single hot link. A hot link (i, k) is treated as a bilateral trade between racks i and k: they jointly decide whether

Algorithm 1. Local Monitoring at Rack *v*

Input: window size Δt, hot threshold θ_{hot}, persistence length K_{hot}
State: sliding-window statistics for each incident link $e = (v, x)$
while DTPE is enabled **do**
  for all incident links $e = (v, x)$ **do**
    update $\bar{\rho}_e(\Delta t)$ using new byte counters
    if $\bar{\rho}_e(\Delta t)$ has exceeded θ_{hot} for K_{hot} consecutive monitoring windows **then**
      mark e as a hot link candidate
  sleep for one monitoring interval

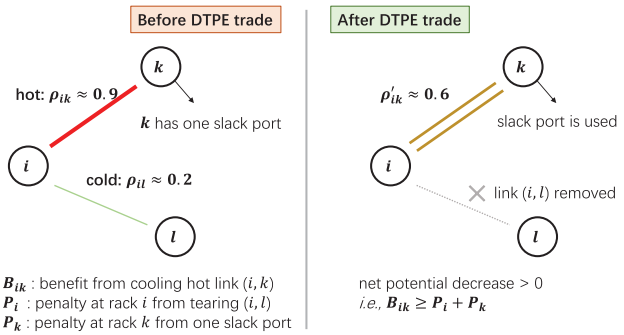

Fig. 4. Illustration of a bilateral DTPE trade. To mitigate congestion on the hot link (i, k), rack i tears down a cold link (i, ℓ) while rack k uses a slack port. The trade proceeds because the congestion benefit (B_{ik}) outweighs the combined port costs $(P_i + P_k)$.

it is worth adding one more optical circuit on (i, k), given the expected reduction in congestion and the cost of freeing one additional port at each endpoint. Figure 4 illustrates the simplest case.

1. Fluid Model for Parallel Circuits

At the timescale of reconfiguration, we adopt a simple fluid model. Traffic on link (i, k) is assumed to be evenly spread across all parallel optical circuits, which is consistent with ECMP-style load balancing in the electrical layer. Let m_{ik} be the current number of full-duplex circuit pairs on (i, k), so the capacity in each direction is $C_{\text{cur}} = m_{ik}C$. Let ρ_{ik} and ρ_{ki} denote the current utilizations in the two directions. If we add one more circuit, the new utilizations are approximated as

$$\rho'_{ik} = \rho_{ik} \cdot \frac{C_{\text{cur}}}{C_{\text{cur}} + C}, \quad \rho'_{ki} = \rho_{ki} \cdot \frac{C_{\text{cur}}}{C_{\text{cur}} + C}. \tag{7}$$

Conversely, when we remove one circuit from a lightly loaded link (v, x) that currently has $m_{vx} \geq 2$ circuits and utilization ρ_{vx}, the new utilization is approximated as

$$\rho'_{vx} = \rho_{vx} \cdot \frac{C_{\text{cur}}}{C_{\text{cur}} - C}, \tag{8}$$

provided that the resulting ρ'_{vx} and ρ'_{xv} remain below a safety threshold $\theta_{\text{safe}} < 1$. This explicit formula replaces the earlier hand-waving "proportional splitting" description and makes our benefit estimates concrete.

2. Congestion Penalty Function

We quantify congestion on a link using a simple convex penalty function $g(\rho)$ of its utilization $\rho \in [0, 1]$. For example, we can use

$$g(\rho) = \max(0, \rho - \theta_{\text{soft}})^2, \tag{9}$$

where θ_{soft} is a soft threshold below which we consider the link safely loaded. Such convex penalties are common in traffic engineering: they give relatively small penalties to moderately loaded links and much larger penalties to very hot links, roughly capturing the risk to job completion time or tail latency.

3. Benefit of Cooling Link (i, k)

Given the current utilizations ρ_{ik} and ρ_{ki} and the predicted post-trade utilizations ρ'_{ik} and ρ'_{ki}, we define the *benefit* of cooling link (i, k) by one circuit as

$$B_{ik} = [g(\rho_{ik}) - g(\rho'_{ik})] + [g(\rho_{ki}) - g(\rho'_{ki})]. \tag{10}$$

Heavily congested links yield large B_{ik}, while lightly loaded links yield little benefit from adding more capacity.

4. Port Cost for a Rack

To add one circuit on (i, k), each endpoint must contribute one optical port. For rack $v \in \{i, k\}$, this port can come from two sources. First, if v has an unused optical port that was reserved by the slack factor in the ILP baseline, then using this port has negligible cost; we model this as a small constant $C_v^{\text{slack}} \approx 0$. Second, if there is no slack port, v can free a port by removing one circuit on some incident link (v, x). Using the fluid model above, this changes the utilizations on (v, x) and (x, v) from ρ_{vx}, ρ_{xv} to ρ'_{vx}, ρ'_{xv}, and increases their congestion penalties. We define the cost of freeing this port as

$$\text{cost}_v(v, x) = [g(\rho_{vx}) - g(\rho'_{vx})] + [g(\rho_{xv}) - g(\rho'_{xv})]. \tag{11}$$

We only consider edges (v, x) for which the resulting utilizations remain below θ_{safe}, so that we do not create new pathological hotspots. Note that links are full-duplex, so $\text{cost}_v(v, x)$ accounts for the penalty change in both directions (v, x) and (x, v) when one circuit pair is removed.

For a given trade T that aims to cool link (i, k), the *port price* that rack v has to pay is the minimum cost of freeing one port:

$$P_v(T) = \min\left(C_v^{\text{slack}}, \min_{(v,x)\text{admissible}} \text{cost}_v(v, x)\right). \tag{12}$$

By construction, $P_v(T)$ depends only on local utilizations and the common penalty function $g(\rho)$.

5. Acceptance Rule for a Bilateral Trade

Given B_{ik} and the port prices $P_i(T)$ and $P_k(T)$ for the two endpoints, DTPE accepts the trade on link (i, k) if

$$B_{ik} \geq P_i(T) + P_k(T). \tag{13}$$

If the inequality holds, the reduction in congestion penalty on (i, k) is at least as large as the combined penalty increase

Algorithm 2. Handle Hot Link (i, k) at Racks i and k

Input: hot link (i, k)
i and k exchange current utilization on (i, k) and local summaries needed to approximate ρ'_{ik}, ρ'_{ki} and port prices
Each rack computes B_{ik} using Eq. (10)
Each rack $v \in \{i, k\}$ computes its port price $P_v(T)$ using Eq. (12)
if $B_{ik} \geq P_i(T) + P_k(T)$ **then**
  i and k agree to perform the trade
  Each rack $v \in \{i, k\}$ frees one port (either via a slack port or by tearing down the cheapest admissible edge)
  i and k jointly request the OCS to add one circuit on (i, k)
  mark link (i, k) as frozen for a hold-down interval T_{freeze}
else
  Trade is rejected; do nothing for this hot link

from freeing one port at each endpoint. Racks i and k then select the cheapest way to free a port [either using a slack port or tearing down the admissible edge that minimizes Eq. (11)] and jointly request the OCS to add one more circuit on (i, k). If the inequality does not hold, DTPE does not modify the topology for this particular hot link, and the burst is left to be absorbed by the long-term ILP baseline and higher-layer mechanisms. This acceptance rule is uniform. We do not need separate case distinctions for "who has slack" or "who must tear down which edge"; all such situations are implicitly encoded in the port prices $P_i(T)$ and $P_k(T)$. Algorithm 2 outlines the end-to-end procedure for handling a detected hot link.

E. Multiple Hot Links and Transit Racks

So far, we have considered a single hot link in isolation. In a real network, a rack may sit on multiple hot links at once. This is especially common for transit racks, which see both incoming and outgoing congestion. We deliberately do not introduce a special multi-party negotiation protocol. Instead, we treat every hot link as a candidate bilateral trade and let each rack decide where to spend its limited port budget.

1. Net Gain of a Trade

For each candidate trade T on hot link (p, q), we define its *net gain* as

$$\text{Gain}(T) = B_{pq} - \big(P_p(T) + P_q(T)\big). \tag{14}$$

A trade with $\text{Gain}(T) < 0$ is rejected by rule Eq. (13). Among the trades that satisfy $\text{Gain}(T) \geq 0$, larger values of $\text{Gain}(T)$ correspond to larger reductions in the congestion penalty.

2. Rack-Local Policy: Highest-Gain-First

Consider a rack v that appears as an endpoint in several candidate trades within a short time window [for example, (ℓ, v) and (v, k) are both hot]. Rack v computes its contribution $P_v(T)$ to each trade and thus can evaluate $\text{Gain}(T)$ for all trades that involve it. We adopt a simple rack-local policy: (i) rack v maintains at most one *active trade* at a time (either initiated by v or involving v as the other endpoint); (ii) among all candidate trades with $\text{Gain}(T) \geq 0$, v agrees to participate in the trade with the largest $\text{Gain}(T)$, provided it is not already committed to another trade; and (iii) for the remaining trades, v replies with a temporary rejection and the initiators may retry after a small random backoff. From a system perspective, this is a "highest net benefit first" rule. For simple endpoint pairs, it reduces to the bilateral acceptance rule in Section 3.D. For transit racks, it resolves the contention between multiple hot links competing for the same finite port budget, without any special multi-party negotiation protocol.

3. Post-Reconfiguration Hold-Down and Oscillation Control

To avoid complicated states and oscillatory behavior, we impose two additional constraints. First, each rack only processes one active trade at a time, which bounds the state and simplifies reasoning about interactions. Second, both endpoints of a link maintain a per-link hold-down timer. After any topology change that adds or removes a circuit on (i, k), the link enters a frozen state for a fixed interval T_{freeze}. During this interval, (i, k) is not eligible for new trades, even if it appears hot again. This simple mechanism prevents ping-pong behavior where the same pair of racks repeatedly trade capacity back and forth.

When a trade is rejected because some rack is already engaged in a more beneficial trade, the initiator waits for a random backoff interval before reconsidering the same hot link. Intuitively, each accepted trade strictly decreases a congestion-based potential (such as the sum of per-link penalties), and trades are prioritized by their net gain. Together with the hold-down timer, we therefore expect DTPE to move the system toward less congested configurations without large oscillations. A full convergence proof is outside the scope of this work; in Section 4, we report empirical evidence that we did not observe pathological oscillations in our simulations. Algorithm 3 summarizes the local negotiation and tie-breaking logic at each rack.

Computational cost. DTPE is lightweight: per monitoring cycle, each rack scans incident links in $O(d_i)$ time (Algorithm 1), and handling a hot link enumerates at most a constant number of candidate rewires bounded by the local degree budgets (Algorithms 2 and 3). Since $d_i \leq d_i^{\max}$ is hardware-limited, DTPE's per-rack computation is $O(1)$ with respect to the network size N and does not block the control loop.

Algorithm 3. Negotiation and Tie Breaking at Rack v

State: set $\mathcal{T}_v$ of candidate trades involving v
State: flag `busy` indicating whether v is in an active trade
if `busy` = `true` **then**
  temporarily reject all new trade proposals involving v
  return
remove from $\mathcal{T}_v$ any trades that would touch a link currently in hold-down at v
compute $\text{Gain}(T)$ for all $T \in \mathcal{T}_v$
let $T^\star$ be the trade with the largest $\text{Gain}(T)$
if $\text{Gain}(T^\star) \geq 0$ **then**
  agree to participate in $T^\star$ and set `busy` $\leftarrow$ `true`
  reply negatively to other trades in $\mathcal{T}_v$
else
  reject all trades in $\mathcal{T}_v$

4. EVALUATION

A. Simulation Setup

We evaluate DTPE via a time-slotted simulator, where each slot corresponds to one monitoring window during which link utilization is measured and (if enabled) DTPE may apply a bounded number of local trades. Unless otherwise stated, each data point is averaged over 40 independent runs with error bars indicating 95% confidence intervals. We consider network sizes $N \in \{8,16,32,64\}$ with a per-rack degree budget $d_{\max}(N)$ (port budget), scaled with N as

$$d_{\max}(N) = \text{round}\left(d_0 \left(\frac{N-1}{N_0-1}\right)^{\alpha_d}\right), \tag{15}$$

with $(N_0, d_0, \alpha_d) = (32, 14, 0.5)$ unless otherwise stated. This sub-linear scaling ($\alpha_d = 0.5$) captures the practical constraint where available optical degrees often grow slower than the network size N. It ensures that the evaluation maintains a consistent sparsity regime across scales, rather than assuming an idealized linear expansion of port budgets. We reserve a fraction σ of each rack's degree budget as slack ports, using upward rounding so that a nonzero σ always provides at least one slack port:

$$d_{\text{slack}} = \begin{cases} 0, & \sigma = 0, \\ \max(1, \lceil \sigma d_{\max} \rceil), & \sigma > 0. \end{cases} \tag{16}$$

Unless otherwise stated, we set $\sigma = 0.05$.

Traffic demands are generated from a skewed (clustered) traffic matrix (TM) model to reflect the common non-uniformity in practical DCNs, where a small fraction of rack pairs carries a large share of the total demand. [We quantify the skewness by the demand shares of top-k rack pairs (e.g., top-10/top-20/top-50) and keep comparable skewness levels across different N in the scalability study. The motivation is to avoid the degeneracy under $d_{\max} \ll N$ that can arise with uniformly dense all-to-all demands, where most ports become structurally pinned and the feasible donor space for temporary reconfiguration is overly compressed, potentially distorting the evaluation of DTPE's effectiveness on burst hotspots.]

To focus on DTPE's adaptability under comparable initial conditions rather than comparing baseline solvers themselves, we apply a baseline-aware calibration: for each baseline planning direction, we scale the input TM so that its planned topology starts from the same initial load level (initial MLU $= 0.5$ at $t = 0$). We superimpose bursty demand on top of the base TM using a parametric burst generator characterized by (i) a burst arrival rate (mean number of new bursts initiated per monitoring slot), (ii) a burst duration (in slots), (iii) a burst intensity (a scaling factor for the extra demand added to hotspot rack pairs relative to the base TM), and (iv) a drift/aggregation parameter α that controls how concentrated/persistent burst hotspots are over time. Importantly, DTPE does not need to attribute a burst to any particular timescale controller; it simply reacts to persistent hotspots it observes, independent of whether a burst is short or long-lived. In the scalability experiment, burst statistics are scaled with N to ensure that representative burst hotspots remain observable under each network size, enabling a meaningful assessment of DTPE under the corresponding $d_{\max}(N)$ constraints.

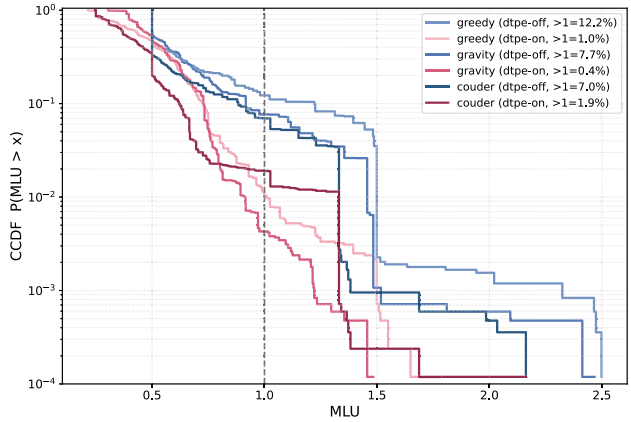

Fig. 5. CCDF of MLU under bursty traffic with/without DTPE for three baseline planning directions (greedy, gravity, and Couder [16]).

The burst generator parameters are calibrated against the Alibaba cluster trace [23], a publicly available production trace collected over 13 days from over ten thousand bare-metal nodes, from which we extracted key statistics including top-k demand concentration, peak burst intensity, and hotspot persistence. The resulting default parameters—burst arrival rate $= 1.2$, burst intensity $= 2.2$, burst duration $= 4$ slots, and $\alpha = 0.65$—are used in the main tail-risk evaluation [Figs. 5 and 9 (below)]. The parametric sweep experiments (Figs. 6–8) systematically vary individual parameters across ranges that encompass the trace-derived values, providing broader coverage of realistic operating conditions.

We compare DTPE under three baseline planning directions. The first two—"greedy" and "gravity"—represent two widely used families of lightweight traffic-aware topology solvers: a demand-sorting/incremental construction style and a gravity-style demand inference and matching style, as adopted in prior OCS/DCN planning studies [1–3,19]. These two directions are retained as representatives of their respective algorithm families; our goal is not to compare planners, but to test DTPE's robustness across qualitatively different planned topologies. The third baseline is Couder [16], a robust quasi-static topology engineering method that represents a stronger planning approach. By including Couder, we evaluate whether DTPE remains effective on top of a higher-quality baseline, and examine the complementarity between fine-timescale distributed perturbation and robust coarse-timescale planning. Throughout, DTPE uses the same hot-link detection mechanism and applies temporary reconfiguration to mitigate emerging hotspots.

B. Main Results: Tail-Risk Reduction

Figure 5 reports the complementary cumulative distribution function (CCDF) of raw MLU measurements (without smoothing) for a network of size $N = 32$ collected over 1200 slots. The results compare the performance with and without DTPE for three baseline planning directions: greedy, gravity, and Couder [16]. DTPE consistently reduces the tail of the

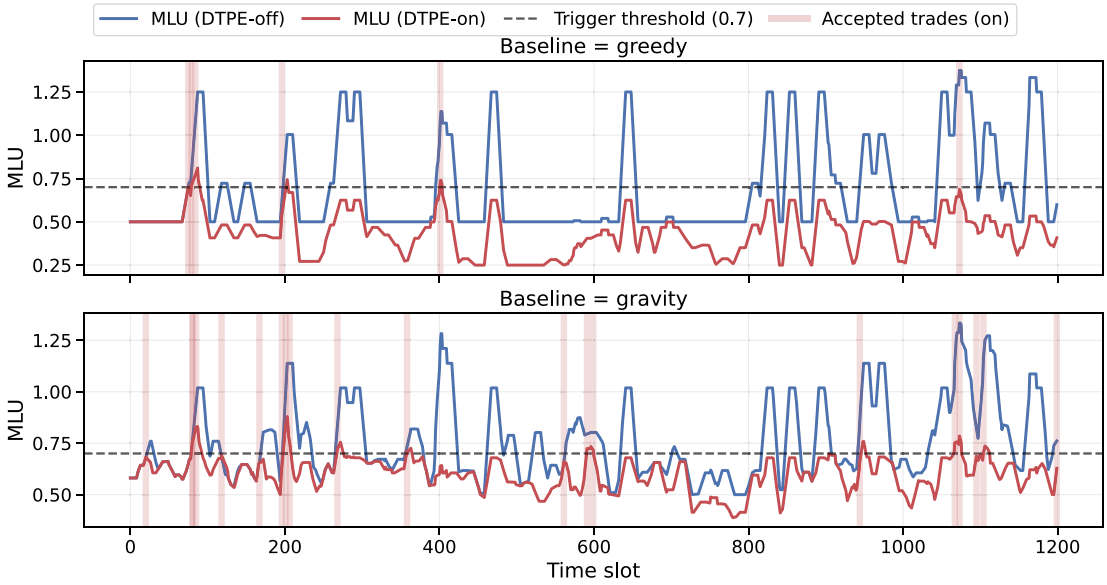

Fig. 6. Representative time series of MLU under bursty traffic. Vertical bars indicate timestamps of accepted DTPE trades.

MLU distribution across all three baselines, yielding orders-of-magnitude reduction in Pr(MLU > 1) in each case. Notably, Couder already provides a stronger baseline with lower initial overload probability; DTPE further reduces this residual risk substantially, demonstrating that fine-timescale distributed perturbation remains effective and complementary even on top of a robust coarse-timescale planner. These results confirm that DTPE's gains persist across qualitatively different planned topologies.

C. Dynamics and Interpretability

To illustrate the dynamic behavior of DTPE, Fig. 6 plots a representative time series of MLU for a network of $N = 32$. To improve readability, the MLU curves are smoothed using a nine-slot moving average. Crucially, Fig. 6 also marks the timestamps of *accepted trades* (shown as vertical bars), indicating moments where DTPE successfully negotiated a topology update. Each slot corresponds to one monitoring cycle. Without DTPE, burst arrivals drive rapid spikes in MLU that can persist over multiple slots. With DTPE, once hot-links are detected, temporary reconfigurations are applied to divert load away from hotspots, flattening the peaks and accelerating recovery. The time-domain view provides an interpretable explanation for the CCDF improvements in Fig. 5: DTPE reduces both the height and the duration of burst-induced congestion episodes.

D. Sensitivity to Burst Drift/Aggregation (α Sweep)

We next study DTPE's sensitivity to the burst drift/aggregation parameter α in Fig. 7. Larger α yields more concentrated and persistent hotspots, whereas smaller α produces more dispersed and rapidly drifting hotspots. Figure 7 reports two key metrics: the overload probability Pr(MLU > 1) on the left axis and the frequency of DTPE interventions (accepted temporary reconfigurations per 100 slots) on the right axis. DTPE

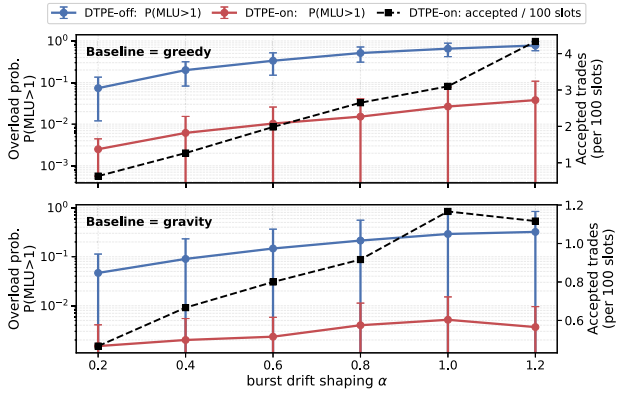

Fig. 7. Sensitivity to burst drift/aggregation parameter α, reporting overload probability Pr(MLU > 1) and DTPE activity.

remains effective across a wide range of α, while the magnitude of improvement naturally depends on how structured and persistent the hotspots are; more persistent hotspots tend to provide clearer opportunities for temporary reconfiguration to meaningfully reduce tail risk.

E. Sensitivity to Reserved Slack (σ Sweep)

We next examine how DTPE responds to different reserved slack levels; i.e., we keep the baseline TM fixed (calibrated at $\sigma = 0$) and increase σ only to reserve per-rack ports according to Eq. (16), which tightens the baseline degree budget and leaves less usable capacity for static planning. Figure 8 reports the resulting overload probability Pr(MLU > 1). As σ increases, both greedy and gravity baselines with DTPE disabled become more overload-prone because reserving ports reduces the effective connectivity available to carry burst traffic, making tail events harder to absorb with a fixed topology. In contrast, enabling DTPE consistently suppresses overload events across all tested σ values by exploiting the reserved slack

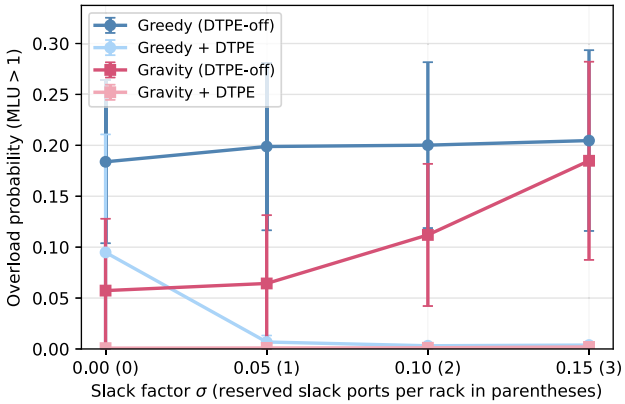

Fig. 8. Overload probability (MLU > 1) versus slack factor σ. Parentheses on the x axis indicate the resulting reserved slack ports per rack.

ports as temporary "rewiring headroom," reallocating degree budget from cooler links to alleviate emerging hotspots. The results confirm that once slack reservation starts tightening budgets, the baseline planner (DTPE-off) degrades while DTPE maintains near-zero overload probability, highlighting DTPE's robustness when the static planner is forced to operate with reduced effective port budgets.

F. Scalability with Network Size

Figure 9 evaluates the scalability of DTPE across network sizes $N \in \{8,16,32,64\}$, with degree budgets scaled according to Eq. (15). We define the performance benefit as the relative percentage reduction in MLU, calculated as $(\mathrm{MLU}_{\mathrm{off}} - \mathrm{MLU}_{\mathrm{on}})/\mathrm{MLU}_{\mathrm{off}}$. Figure 9 plots the 95$^{\mathrm{th}}$ percentile of the MLU distribution for four baseline planning directions: ILP (shown for $N = 8$ only due to scalability constraints), greedy, gravity, and Couder [16]. DTPE delivers consistent tail-risk reductions across all baselines and network sizes. The attainable gain varies with N and baseline quality for two complementary reasons. First, as the baseline planner improves (e.g., from greedy/gravity to Couder, or from heuristic to ILP), the residual burst-induced mismatch after planning decreases, so the marginal gain achievable by

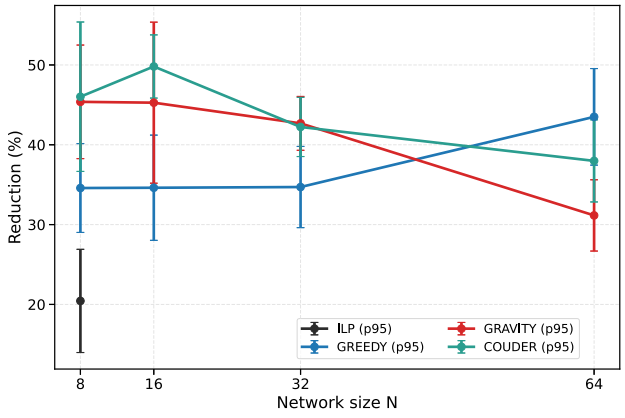

Fig. 9. Scalability of DTPE performance gains (reduction in MLU $p95$ tail statistic) across network sizes N, under four baseline planning directions. An ILP-based oracle reference is shown for $N = 8$ only.

DTPE also decreases; this reflects the complementary nature of DTPE rather than a limitation. Second, at larger N, greedy and gravity exhibit a crossover in their relative gains due to differences in how each planner handles the increasing sparsity regime: gravity's matching-style construction becomes less effective at large N under the sub-linear degree budget, leaving less residual hotspot structure for DTPE to exploit, while greedy's incremental construction retains more localized hotspots that DTPE can address. Overall, DTPE provides meaningful robustness improvements across a broad range of network scales and baseline planning approaches under realistic $d_{\max}(N)$ constraints.

G. Trace-Driven Replay Validation of Reconfiguration Benefits

To validate the physical feasibility of the proposed reconfiguration actions, we conduct a trace-driven replay on a four-rack optical circuit switching setup. Rather than implementing the full distributed DTPE control plane and online agent negotiation, we precompute DTPE's topology-change schedule from the observed workload trace and replay these reconfigurations using a controller script synchronized with the traffic workload.

This design isolates the data-plane effect of topology adaptation from implementation-specific control-plane jitter and allows us to quantify the end-to-end benefit of DTPE's reconfiguration actions under real application traffic.

Figure 10 shows the normalized JCT results from our testbed under three representative distributed machine learning communication patterns: parameter server (PS), distributed data parallel (DDP), and peer-to-peer (P2P). For each pattern, we normalize the JCT to a burst-free baseline scenario (leftmost bars). The "without DTPE" scenario (middle bars) shows that unhandled bursts cause severe performance degradation, inflating JCT by approximately 40%–50% due to persistent congestion on static links. In contrast, the "with DTPE" scenario (rightmost bars) demonstrates that applying the negotiated reconfigurations effectively mitigates this inflation. Despite the transient throughput dips caused by the physical reconfiguration delay, the optimized topology capacity successfully absorbs the burst traffic, bringing the JCT back to

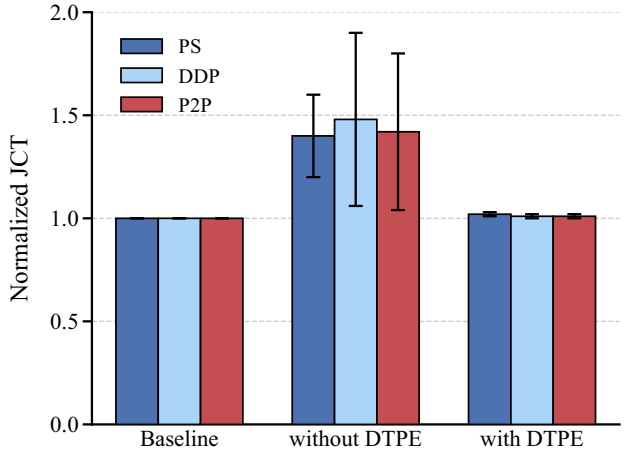

Fig. 10. Trace-driven replay results: normalized job completion time (JCT) under three communication patterns (PS/DDP/P2P).

near-baseline levels (within 2%−3% overhead). These replay results provide supporting evidence that mitigating tail congestion at the topology layer can translate into tangible end-to-end performance benefits on real workloads. While the physical testbed is small, the replay qualitatively validates the *data-plane feasibility* of executing DTPE-style reconfigurations: the optical hardware can keep up with the reconfiguration frequency dictated by DTPE without incurring excessive performance loss during switching; algorithmic gains at scale are evaluated via our simulation results.

5. CONCLUSION

This paper studied the interplay between a centralized baseline planner and a fast-time-scale distributed layer for mitigating burst-induced tail risk under per-rack degree constraints in optical DCNs. We presented DTPE, a distributed topology perturbation engine that performs safe, local topology trades using reserved slack ports, without creating new adjacencies.

Extensive simulations under representative skewed traffic show that DTPE consistently suppresses overload events and reduces the MLU tail across different baseline planners and network sizes. A trace-driven replay further provides qualitative validation that DTPE-style reconfiguration actions can translate into end-to-end application-level benefits on real traffic, while highlighting the transient performance dips during switching.

DTPE is intentionally modular: system deployments can incorporate technology-specific reconfiguration delays and richer monitoring signals while keeping the same local trading interface and acceptance rule.

REFERENCES

1. L. Xu, A. Singh, and Y. Zhang, "Optically interconnected data center networks," in *Optical Fiber Communication Conference (OFC)* (2012), paper OW3J.3.
2. K. Chen, A. Singla, A. Singh, *et al.*, "OSA: an optical switching architecture for data center networks with unprecedented flexibility," IEEE/ACM Trans. Netw. **22**, 498–511 (2014).
3. K.-T. Foerster and S. Schmid, "Survey of reconfigurable data center networks: enablers, algorithms, complexity," ACM SIGACT News **50**, 62–79 (2019).
4. M. N. Hall, K. T. Foerster, S. Schmid, *et al.*, "A survey of reconfigurable optical networks," Opt. Switch. Netw. **40**, 100616 (2021).
5. N. Farrington, G. Porter, S. Radhakrishnan, *et al.*, "Helios: a hybrid electrical/optical switch architecture for modular data centers," in *Proceedings of the ACM SIGCOMM* (2010), pp. 339–350.
6. G. Wang, D. G. Andersen, M. Kaminsky, *et al.*, "C-through: part-time optics in data centers," in *Proceedings of the ACM SIGCOMM* (2010), pp. 327–338.
7. H. Yang and Z. Zhu, "Topology configuration scheme for accelerating coflows in a Hyper-FleX-LION," J. Opt. Commun. Netw. **14**, 805–814 (2022).
8. H. Yang and Z. Zhu, "Traffic-aware configuration of all-optical data center networks based on Hyper-FleX-LION," IEEE/ACM Trans. Netw. **32**, 2675–2688 (2024).
9. S. Jain, A. Kumar, S. Mandal, *et al.*, "B4: experience with a globally-deployed software defined WAN," in *Proceedings of the ACM SIGCOMM* (2013), pp. 3–14.
10. C.-Y. Hong, S. Kandula, R. Mahajan, *et al.*, "SWAN: software-driven WAN," in *Proceedings of the ACM SIGCOMM* (2013), pp. 52–63.
11. H. Yang, X. Pan, S. Zhao, *et al.*, "On the bilevel optimization for remapping virtual networks in an HOE-DCN," IEEE Trans. Netw. Serv. Manag. **19**, 1274–1286 (2022).
12. L. Gong and Z. Zhu, "Virtual optical network embedding (VONE) over elastic optical networks," J. Lightwave Technol. **32**, 450–460 (2014).
13. J. Liu, W. Lu, F. Zhou, *et al.*, "On dynamic service function chain deployment and readjustment," IEEE Trans. Netw. Serv. Manag. **14**, 543–553 (2017).
14. M. Chowdhury, Y. Zhong, and I. Stoica, "Efficient coflow scheduling with Varys," in *Proceedings of the ACM SIGCOMM* (2014), pp. 443–454.
15. M. Shafiee and J. Ghaderi, "Scheduling coflows in datacenter networks: improved bound for total weighted completion time," ACM SIGMETRICS Perform. Eval. Rev. **44**, 29–31 (2017).
16. M. Y. Teh, S. Zhao, P. Cao, *et al.*, "Enabling quasi-static reconfigurable networks with robust topology engineering," IEEE/ACM Trans. Netw. **31**, 1056–1070 (2023).
17. X. Xie, B. Tang, X. Chen, *et al.*, "P4INC-AOI: all-optical interconnect empowered by in-network computing for DML workloads," IEEE/ACM Trans. Netw. **33**, 1236–1251 (2025).
18. Q. Lv, Y. Zhang, S. Zhang, *et al.*, "On the TPE design to efficiently accelerate hitless reconfiguration of OCS-based DCNs," IEEE J. Sel. Areas Commun. **43**, 1780–1792 (2025).
19. Y. Tang, T. Yuan, B. Liu, *et al.*, "Effective *-flow schedule for optical circuit switching based data center networks: a comprehensive survey," Comput. Netw. **197**, 108321 (2021).
20. P. Cao, S. Zhao, D. Zhang, *et al.*, "Threshold-based routing-topology co-design for optical data center," IEEE/ACM Trans. Netw. **31**, 2870–2885 (2023).
21. P. Cao, S. Zhao, M. Y. Teh, *et al.*, "TROD: evolving from electrical data center to optical data center," in *Proceedings of the 2021 IEEE 29th International Conference on Network Protocols (ICNP)* (2021).
22. Y. Mao, Q. Zhai, Z. Yao, *et al.*, "ATRO: a fast solver-free algorithm for topology and routing optimization of reconfigurable datacenter networks," arxiv (2025).
23. Alibaba Group, "Alibaba cluster trace program," GitHub (2022), https://github.com/alibaba/clusterdata.